

Competitive Programming

Section 7 Editorial: Foundation mixed contest

Competitive Programming Course

2026-07-28

Table of contents

1	Category coverage	4
2	How to read this editorial	5
3	Problem A: Inventory Cleanup	6
3.1	Restatement	6
3.2	Important constraints and traps	6
3.3	Observations	6
3.4	Sample walk-through	6
3.5	Algorithm	7
3.6	Correctness	7
3.7	Complexity	7
3.8	C++ solution	7
3.9	Python solution	8
3.10	Common wrong answers	8
4	Problem B: Peak Load	9
4.1	Restatement	9
4.2	Important constraints and traps	9
4.3	Observations	9
4.4	Sample walk-through	9
4.5	Algorithm	10
4.6	Correctness	10
4.7	Complexity	10
4.8	C++ solution	10
4.9	Python solution	11
4.10	Common wrong answers	12
5	Problem C: Best Study Streak	13
5.1	Restatement	13
5.2	Important constraints and traps	13
5.3	Observations	13
5.4	Sample walk-through	13
5.5	Algorithm	13
5.6	Correctness	14
5.7	Complexity	14
5.8	C++ solution	14
5.9	Python solution	15
5.10	Common wrong answers	16
6	Problem D: Balanced Break	17
6.1	Restatement and traps	17
6.2	Trace and algorithm	17
6.3	Correctness proof	17
6.4	Complexity and language notes	17

6.5	Full C++ solution	17
7	Problem E: Circular Patrol	19
7.1	Restatement and constraint reading	19
7.2	State and representation	19
7.3	Algorithm	19
7.4	Correctness proof	19
7.5	Complexity	20
7.6	Implementation notes and common mistakes	20
7.7	Full C++ solution	20
7.8	Full Python solution	21
7.9	Tests that matter	21
8	Contest testing strategy	22
9	What to carry forward	23

1 Category coverage

Category	Problem
simulation and brute force	E. Circular Patrol
counting and maps	A. Inventory Cleanup
prefix sums and difference arrays	B. Peak Load; D. Balanced Break
sorting	A. Inventory Cleanup
two pointers and sliding windows	C. Best Study Streak

Circular Patrol is the local blind-practice representative for explicit state simulation and is included by this section's checker. Balanced Break remains useful exhaustive candidate checking with a constant-time prefix update.

2 How to read this editorial

This is a mixed-contest editorial. For each problem, first identify the clue that selects the technique. Then read the proof and implementation details.

The goal is not only to know the solution after seeing it. The goal is to notice why this solution should have been considered during the contest.

3 Problem A: Inventory Cleanup

3.1 Restatement

Delete the fewest items so that at most k distinct colors remain.

3.2 Important constraints and traps

Colors can be large or negative, so use a map/dictionary. If $k = 0$, all items must be deleted. If the number of distinct colors is already at most k , the answer is zero.

The actual color labels do not matter after counting. Only frequencies matter.

3.3 Observations

To reduce the distinct-color count, deleting only some items of a color is not useful unless the whole color disappears. A color remains distinct as long as at least one item of that color remains.

Therefore each deleted color group costs its frequency. If there are d distinct colors and we need at most k , then we must remove:

```
max(0, d - k)
```

entire color groups. To minimize deletions, remove the groups with smallest frequencies.

3.4 Sample walk-through

Colors:

```
1 1 2 2 3 4 4
```

Frequencies:

color	count
1	2
2	2
3	1
4	2

There are 4 distinct colors. If $k = 2$, remove 2 color groups. The smallest group sizes are 1 and 2, so the answer is 3.

3.5 Algorithm

```
count color frequencies
sort the frequency values ascending
remove_groups = max(0, distinct_count - k)
answer = sum of the remove_groups smallest frequencies
print answer
```

3.6 Correctness

Any solution that leaves at most k colors must completely delete at least $d - k$ color groups, where d is the number of distinct colors. Deleting only part of a color group does not reduce the distinct count and can only increase the number of deletions, so an optimal solution deletes whole groups.

Each deleted group has cost equal to its frequency. Among all choices of $d - k$ groups, the minimum total cost is obtained by choosing the smallest $d - k$ frequencies. The algorithm does exactly that after sorting. Therefore the printed number is minimum.

3.7 Complexity

Counting is $O(n)$ expected time. Sorting the distinct frequencies is $O(d \log d)$, where $d \leq n$. Memory is $O(d)$.

3.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, k;
        cin >> n >> k;
        unordered_map<int, int> freq;
        freq.reserve(2 * n + 1);
        for (int i = 0; i < n; ++i) {
            int color;
            cin >> color;
            ++freq[color];
        }
    }
}
```

```

vector<int> counts;
for (auto [color, count] : freq) {
    counts.push_back(count);
}
sort(counts.begin(), counts.end());
int remove_groups = max(0, static_cast<int>(counts.size()) - k);
int answer = 0;
for (int i = 0; i < remove_groups; ++i) {
    answer += counts[i];
}
cout << answer << '\n';
}
return 0;
}

```

3.9 Python solution

```

from collections import Counter
import sys

def min_deletions(values: list[int], k: int) -> int:
    freq = sorted(Counter(values).values())
    remove = max(0, len(freq) - k)
    return sum(freq[:remove])

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    k = data[idx + 1]
    idx += 2
    values = data[idx:idx + n]
    idx += n
    out.append(str(min_deletions(values, k)))
sys.stdout.write("\n".join(out))

```

3.10 Common wrong answers

- Deleting individual rare items without removing whole colors.
- Keeping the most frequent k colors but computing the deleted count incorrectly.
- Mishandling k = 0.
- Using a frequency array for large or negative color values.

4 Problem B: Peak Load

4.1 Restatement

Each job adds load w to every integer time in $[1, r]$. Find the maximum total load and the earliest time when it occurs.

4.2 Important constraints and traps

The sum of $n + m$ is bounded, so an $O(n + m)$ difference-array solution is intended. Loads may be negative, so initialize the best load carefully; it may also be negative.

The tie rule is earliest time. A left-to-right scan should update only on strictly greater load.

4.3 Observations

This is a range-add problem. For each job:

```
diff[1] += w
diff[r + 1] -= w
```

Then scan times from 1 to m , maintaining the current load.

4.4 Sample walk-through

Jobs:

```
[1,3] +5
[2,5] +4
[6,6] +10
```

Loads:

time	load
1	5
2	9
3	9
4	4
5	4
6	10

The maximum is 10 at time 6.

4.5 Algorithm

```
diff = zeros of length m + 2
for each job l, r, w:
    diff[l] += w
    diff[r + 1] -= w
cur = 0
best = -infinity
best_time = 1
for time from 1 to m:
    cur += diff[time]
    if cur > best:
        best = cur
        best_time = time
print best, best_time
```

4.6 Correctness

For a single job $[l, r, w]$, the running sum of `diff` includes w at every time t with $l \leq t \leq r$, and includes zero net contribution outside that range. By linearity of addition, after all jobs are marked, the running sum at time t equals the total load of all jobs covering t .

The scan visits times in increasing order. Whenever it sees a load strictly larger than all previous loads, it records that load and time. If the same maximum appears later, the scan does not update, so the earliest time remains stored. Therefore the output is the maximum load and its earliest occurrence.

4.7 Complexity

Each job is processed in $O(1)$, and the scan is $O(m)$. Memory is $O(m)$.

4.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, m;
        cin >> n >> m;
```

```

vector<long long> diff(m + 2, 0);
for (int i = 0; i < n; ++i) {
    int l, r;
    long long w;
    cin >> l >> r >> w;
    diff[l] += w;
    diff[r + 1] -= w;
}
long long cur = 0;
long long best = LLONG_MIN;
int best_time = 1;
for (int time = 1; time <= m; ++time) {
    cur += diff[time];
    if (cur > best) {
        best = cur;
        best_time = time;
    }
}
cout << best << ' ' << best_time << '\n';
}
return 0;
}

```

4.9 Python solution

```

import sys

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    m = data[idx + 1]
    idx += 2
    diff = [0] * (m + 2)
    for _ in range(n):
        l = data[idx]
        r = data[idx + 1]
        w = data[idx + 2]
        idx += 3
        diff[l] += w
        diff[r + 1] -= w
    cur = 0
    best = None
    best_time = 1
    for time in range(1, m + 1):
        cur += diff[time]

```

```
    if best is None or cur > best:
        best = cur
        best_time = time
    out.append(f"{best} {best_time}")
sys.stdout.write("\n".join(out))
```

4.10 Common wrong answers

- Updating on $\geq$ and returning the latest maximum instead of earliest.
- Initializing best to 0 when all loads can be negative.
- Forgetting the $r + 1$ marker.
- Applying every job to every time directly.

5 Problem C: Best Study Streak

5.1 Restatement

Find the longest contiguous subarray with sum at most k in a nonnegative array. If several longest streaks exist, print the earliest 1-indexed start. If no nonempty streak is valid, print 0.

5.2 Important constraints and traps

Values are nonnegative, so sliding window applies. The tie rule requires start tracking. The answer may be zero if every value is greater than k .

5.3 Observations

Use the same window as Section 6. After shrinking, `[left, right]` is the longest valid window ending at `right`. Compare it with the best answer so far.

Because `right` increases from left to right, equal-length windows often appear in start order, but the safest implementation still writes the tie comparison explicitly.

5.4 Sample walk-through

Values:

```
5 1 1 1 1 5
```

Limit 5.

The best streak is:

```
1 1 1 1
```

starting at position 2, length 4. The single 5 values are valid but shorter.

5.5 Algorithm

```

left = 0
sum = 0
best_len = 0
best_start = 0
for right from 0 to n - 1:
    sum += a[right]
    while sum > k:
        sum -= a[left]
        left += 1
    length = right - left + 1
    start = left + 1
    if length is better than current answer:
        update answer
print best_len, best_start

```

5.6 Correctness

After shrinking for a fixed **right**, the current window has sum at most **k**. Because all values are nonnegative, any window ending at **right** and starting before **left** has sum greater than **k**; otherwise the shrink loop would have stopped earlier. Thus the current window is the longest valid window ending at **right**.

The algorithm compares this best window for each right endpoint against the global best, using length first and start position second. Every valid window has some right endpoint, and for that endpoint it is no longer than the window considered by the algorithm. Therefore the final stored answer is the maximum length, with earliest start among ties.

If no nonempty valid window exists, **best_len** remains 0 and the algorithm prints 0 0, as required.

5.7 Complexity

Both pointers move only forward. Time is $O(n)$, memory is $O(1)$.

5.8 C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;

```

```

    long long limit;
    cin >> n >> limit;
    vector<long long> values(n);
    for (long long &value : values) cin >> value;

    int left = 0;
    long long total = 0;
    int best_len = 0;
    int best_start = 0;
    for (int right = 0; right < n; ++right) {
        total += values[right];
        while (left <= right && total > limit) {
            total -= values[left];
            ++left;
        }
        int length = right - left + 1;
        int start = left + 1;
        if (length > best_len || (length == best_len && length > 0 && start < best_start)) {
            best_len = length;
            best_start = start;
        }
    }
    cout << best_len << ' ' << best_start << '\n';
}
return 0;
}

```

5.9 Python solution

```

import sys

def solve(values: list[int], limit: int) -> tuple[int, int]:
    left = 0
    total = 0
    best_len = 0
    best_start = 0
    for right, value in enumerate(values):
        total += value
        while left <= right and total > limit:
            total -= values[left]
            left += 1
        length = right - left + 1
        start = left + 1
        if length > best_len or (length == best_len and length > 0 and start < best_start):
            best_len = length
            best_start = start
    return best_len, best_start

```

```

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    limit = data[idx + 1]
    idx += 2
    values = data[idx:idx + n]
    idx += n
    length, start = solve(values, limit)
    out.append(f"{length} {start}")
sys.stdout.write("\n".join(out))

```

5.10 Common wrong answers

- Printing the end position instead of the start.
- Forgetting 0 0 when no nonempty window is valid.
- Applying the method to arrays with negative values.
- Updating the answer before shrinking.

6 Problem D: Balanced Break

6.1 Restatement and traps

Choose a nonempty-prefix/nonempty-suffix split minimizing the absolute difference of their sums. The smallest split index wins ties. With total n up to 200000, recomputing both sums for every split is quadratic; signed values mean the running difference need not move monotonically.

6.2 Trace and algorithm

For 3 1 4 1 5, total is 14. Prefix sums at split positions 1,2,3,4 are 3,4,8,9, so differences are 8,6,2,4. The answer is 2 3.

```
total = sum(a); left = 0; best = infinity
for i = 1..n-1:
    left += a[i]
    difference = abs(left - (total-left))
    if difference < best: store difference and i
```

6.3 Correctness proof

After processing index i , `left` is exactly the sum of positions 1 through i , so `total-left` is exactly the suffix sum for that split. The loop evaluates every legal split once. Strict replacement preserves the first index among equal minima. Therefore the stored pair is the required minimum difference and smallest position. $\square$

6.4 Complexity and language notes

Time is $O(n)$ and extra memory is $O(1)$ beyond input storage. C++ needs `long long` for totals and differences; Python integers are exact. Do not stop early when the difference rises: negative values destroy that shortcut.

6.5 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;

    while (t--) {
        int n;
        cin >> n;

        vector<long long> a(n);
        long long total = 0;
        for (long long& x : a) {
            cin >> x;
            total += x;
        }

        long long left = 0;
        long long best = LLONG_MAX;
        int position = 1;

        for (int i = 0; i < n - 1; ++i) {
            left += a[i];
            long long difference = llabs(left - (total - left));

            if (difference < best) {
                best = difference;
                position = i + 1;
            }
        }

        cout << best << ' ' << position << '\n';
    }
}

```

7 Problem E: Circular Patrol

7.1 Restatement and constraint reading

A robot starts at position p on a circle of n positions. Every command moves it exactly one position left or right, with wraparound. We must output both its final position and the number of different positions seen, counting the start.

The total of $n + |\text{commands}|$ over all test cases is at most 200000. This rules out rescanning the walk for every position, but direct simulation is comfortably fast: each command should cause one constant-time state update.

7.2 State and representation

The complete state needed after a prefix of commands is:

```
position = the robot's current zero-based position
seen[x]   = whether position x has appeared at least once
distinct  = the number of true entries in seen
```

Zero-based positions make wraparound convenient. A right move is $(\text{position} + 1) \% n$. A left move could use $(\text{position} - 1 + n) \% n$; the implementation equivalently adds $n - 1$.

7.3 Algorithm

1. Convert p to zero-based indexing.
2. Mark p as seen and initialize `distinct = 1`.
3. For each command:
 - update the position by one step with wraparound;
 - if the new position was unseen, mark it and increment `distinct`.
4. Convert the final position back to one-based indexing and print it with `distinct`.

7.4 Correctness proof

We prove the following invariant after processing any prefix of the command string:

1. `position` is the robot's true position after that prefix;
2. `seen[x]` is true exactly for positions visited during the start and that prefix;
3. `distinct` equals the number of such positions.

Before processing commands, the robot is at `p`, only `p` has been marked, and `distinct` is one, so the invariant holds. For the next command, the modular update performs exactly the stated left or right move, proving condition 1. The reached position is then marked. If it was new, `distinct` is increased once; if it was already present, the count is unchanged. Conditions 2 and 3 therefore remain true. By induction, the invariant holds after all commands, so the two printed values are exactly the requested answers.

7.5 Complexity

For a test case, every command is processed once, so the time is $O(|s|)$. The boolean array uses $O(n)$ memory. Across all test cases, the stated input bound makes the total work $O(200000)$.

7.6 Implementation notes and common mistakes

- Mark the starting position before reading any move.
- Convert back to one-based indexing only when printing.
- In C++, do not write `(position - 1) % n` without first adding `n`; a negative remainder is possible.
- The answer asks for distinct positions, not the number of visits or moves.
- The case `n = 1` requires no special branch when modular arithmetic is used.

7.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--) {
        int n, position;
        string commands;
        cin >> n >> position >> commands;
        --position;
        vector<char> seen(n, false);
        seen[position] = true;
        int distinct = 1;
        for (char command : commands) {
            position = (position + (command == 'R' ? 1 : n - 1)) % n;
            if (!seen[position]) {
                seen[position] = true;
                ++distinct;
            }
        }
    }
}
```

```

        cout << position + 1 << ' ' << distinct << '\n';
    }
}

```

7.8 Full Python solution

```

import sys

def main() -> None:
    tokens = sys.stdin.buffer.read().split()
    tests = int(tokens[0])
    index = 1
    answers = []
    for _ in range(tests):
        n = int(tokens[index])
        position = int(tokens[index + 1]) - 1
        commands = tokens[index + 2].decode()
        index += 3
        seen = {position}
        for command in commands:
            position = (position + (1 if command == "R" else -1)) % n
            seen.add(position)
        answers.append(f"{position + 1} {len(seen)}")
    print("\n".join(answers))

if __name__ == "__main__":
    main()

```

7.9 Tests that matter

n = 1 with several commands -> final 1, distinct 1.
 Start at 1 and move left -> verifies wraparound to n.
 Move around the full circle and return -> distinct n, not |s| + 1.
 Alternate L and R -> many visits but very few distinct positions.
 No repeated position until the final move -> checks the first-visit counter.

The randomized oracle deliberately uses explicit one-based movement rather than the solution's zero-based formula, reducing the chance that both share the same indexing mistake.

8 Contest testing strategy

For a mixed set, each problem should get a different oracle:

- Problem A: brute force over which distinct colors to keep for very small distinct counts, or compare against sorted frequencies.
- Problem B: direct time simulation for small m .
- Problem C: quadratic enumeration of all subarrays.

The local random tests use these simple references. This is not extra polish; it is how you keep contest code honest.

9 What to carry forward

Phase 0 is now complete. The foundation tools are useful only if you can select them under time pressure. The next phase begins with binary search on the answer, where the same habits continue: read constraints, identify monotonicity, prove the predicate, and test edge cases.